

Basic (your first working tool) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b

model_with_tools = ChatOpenAI().bind_tools([add])

response = model_with_tools.invoke("What's 5 + 7?")
print("Requested tool calls:", response.tool_calls)

for call in response.tool_calls:
    if call["name"] == "add":
        result = add.invoke(call["args"])
        print("Real result:", result)
```

Expected output:

```
Requested tool calls: [{'name': 'add', 'args': {'a': 5, 'b': 7}, 'id': 'call_abc1'}]
Real result: 12
```

This version works correctly. It doesn't handle the case where the model calls something other than `add`, or calls nothing at all — fine for a first working version, worth fixing next.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — checked, and wrapped in real functions

```
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b

def get_model_with_tools() -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([add])

def main() -> None:
    model_with_tools = get_model_with_tools()
    response = model_with_tools.invoke("What's 5 + 7?")

    if not response.tool_calls:
        print("The model answered directly, without calling a tool.")
        print(response.content)
        return

    for call in response.tool_calls:
        if call["name"] == "add":
            result = add.invoke(call["args"])
            print(f"Called add({call['args']}) -> {result}")
        else:
            print(f"Unexpected tool requested: {call['name']}")

if __name__ == "__main__":
    main()
```

Expected output:

```
Called add({'a': 5, 'b': 7}) -> 12
```

Difference from Basic: checking `if not response.tool_calls` handles the case where the model just answers directly instead of calling anything — a real possibility even for an obvious math question, and one your code needs to not crash on. Checking `call["name"]` before running it also matters the moment you register a second tool, since you can't assume the first (or only) requested call is the one you expected. Neither of these yet handles more than one tool call arriving in the same response, or sends anything back to the model — that's what Advanced adds.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — the full round trip, handling every tool call in the response

```
from langchain_core.messages import ToolMessage
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def add(a: int, b: int) -> int:
    """Add two integers together and return the sum."""
    return a + b

def get_model_with_tools() -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([add])

def main() -> None:
    model_with_tools = get_model_with_tools()
    question = "What's 5 + 7? Also, separately, what's 3 + 4?"

    response = model_with_tools.invoke(question)
    print(f"Requested {len(response.tool_calls)} tool call(s): {response.tool_calls}")

    if not response.tool_calls:
        print(response.content)
        return

    messages = [{"role": "user", "content": question}, response]

    for call in response.tool_calls:
        if call["name"] != "add":
            print(f"Unexpected tool requested: {call['name']}")
            continue
        result = add.invoke(call["args"])
        print(f"Called add({call['args']}) -> {result}")
        messages.append(ToolMessage(tool_call_id=call["id"], content=str(result)))

    final_response = model_with_tools.invoke(messages)
```

```
print(f"Model's final answer: {final_response.content}")

if __name__ == "__main__":
    main()
```

Expected output:

```
Requested 2 tool call(s): [{'name': 'add', 'args': {'a': 5, 'b': 7}, 'id': 'call_
Called add({'a': 5, 'b': 7}) -> 12
Called add({'a': 3, 'b': 4}) -> 7
Model's final answer: 5 + 7 is 12, and 3 + 4 is 7.
```

Both tool calls get run and both get tagged back with their own `call["id"]` — mixing that up (sending call 1's result tagged as call 2's) would give the model a confusing, wrong-looking transcript, even though every individual number is correct.

Approach 2 — forcing a call with `tool_choice`, instead of just hoping for one

Core Concepts mentions tool-choice settings (`auto` / `required` /forced). For an exercise like this one, where you already know the question needs `add`, forcing the call removes the "what if it just answers directly" branch entirely, instead of defensively coding around it:

```
def get_model_forced_to_call_add() -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools(
        [add], tool_choice="add"
    )

model_with_tools = get_model_forced_to_call_add()
response = model_with_tools.invoke("What's 5 + 7?")
print(response.tool_calls)
```

Expected output:

```
[{'name': 'add', 'args': {'a': 5, 'b': 7}, 'id': 'call_abc123', 'type': 'tool_cal
```

This guarantees a call to `add` specifically — useful when you, the developer, already know what needs to happen next and don't want to leave it to the model's judgment. It's the wrong choice the moment a prompt might *not* need `add` at all (like a genuinely ambiguous question), since forcing a specific tool removes the model's ability to say "actually, no tool is needed here."

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate handles one tool call safely but silently drops any additional ones, and never lets the model see its own tool's result. Approach 1 closes both gaps — looping over every call in the response and completing the round trip with a real final answer. Approach 2 solves a different problem: it removes the "did it even call anything" uncertainty at the source, by telling the model which tool it must use, which only makes sense when you already know the answer to that question yourself.

Which one should you actually write? Approach 1's full round trip (loop over every tool call, tag results back with `ToolMessage`, ask again for the final answer) is the shape every real tool-calling program uses — Doc07's agent loop is this exact pattern, run repeatedly. Reach for Approach 2's `tool_choice` only in the narrow case where you, as the developer, already know exactly which tool must run next — for example, a fixed pipeline step, not open-ended conversation — since forcing a tool removes the model's ability to correctly decide "no tool is needed" for anything that doesn't actually match.